

# 모든 질문에 답하는 마지막 모델은 없다

유재우, KAIST 전산학부

대규모 언어 모델(large language model)의 정확성이 높아질수록 모든 질문에 옳게 답하는 ‘마지막 모델’에 대한 기대도 커진다. 이 글은 그러한 모델의 예측과 반대로 행동하는 프로그램을 구성하는 사고실험으로, 언제나 답하면서 언제나 옳다는 약속이 성립할 수 있는지 묻는다. 이 한계가 개별 결과의 신뢰까지 부정하지는 않는다. OpenAI의 나비에-스토르크스 해결안과 Lean을 통한 증명 검사는 모델의 권위에 기대지 않고 결과의 근거를 확인하는 방법을 보여 준다. 다만 이 신뢰에는 공리, 형식화, 검사기의 정확성이라는 전제가 남는다. 나는 완전한 모델을 기다리기보다 이러한 전제를 드러내고 판단을 보류할 수 있는 체계를 설계해야 한다고 생각하며, 그 과정에 인간이 개입할 자리가 남는지 질문한다.

## 1 특이점을 향한 경쟁

대규모 언어 모델의 환각(AI hallucination)은 인공지능 기술의 급속한 발전에도 여전히 남아 있는 뚜렷한 한계이다. 모델은 그럴듯한 문장을 만들면서도 사실과 다른 내용을 자신 있게 말하며, 우리는 흔히 그 원인을 블랙박스(black box)인 내부 구조에서 찾는다. 그러나 내부를 해석하기 어렵다는 문제와 원리적으로 답할 수 없다는 문제는 구별해야 한다. 전자는 더 나은 분석 기술로 완화할 수 있지만, 후자는 모델의 매개 변수를 모두 들여다보아도 사라지지 않는다. 이 구별을 놓치면 모델이 충분히 커지고 투명해지기만 하면 언젠가 모든 질문에 답할 수 있으리라고 오해하게 된다.

현재의 환각 완화 기법은 마지막 모델을 만들기보다 여러 불완전한 장치를 조합해 오류 확률을 낮춘다. 검색 증강 생성(retrieval-augmented generation; RAG)은 외부 문서를 찾아 답변의 근거로 제공하고, 연쇄 검증(chain-of-verification)은 초안을 바로 믿지 않고 사실 확인용 질문을 따로 만들어 다시 답하게 한다. 한 모델이 답을 생성하고 다른 모델이 이를 비평하게 하거나, 여러 모델이 서로의 추론을 토론하게 하는 방법도 사용된다. 이를 간단히 쓰면 생성 모델  $M_0$ 의 답을 비평 모델  $M_1$ 이 고치고, 필요하면  $M_2$ 가 다시 살피는 구조이다.

$$q \xrightarrow{M_0} a_0 \xrightarrow{M_1} a_1 \xrightarrow{M_2} \dots \xrightarrow{M_n} a_n$$

이러한 사슬은 실제 정확성을 높이지만, 각 모델이 비슷한 학습 자료와 추론 습관을 공유한다면 같은 오류에 함께 동의할 수도 있다. 만장일치는 높은 신뢰도의 근거일 수 있어도 진리의 증명은 아니다. 그럼에도 이 사슬을 계속 연장하고 각 모델의 능력을 키우다 보면, 언젠가 어떤 질문에도 옳게 답하는 하나의 모델에 도달할 것처럼 보인다.

나는 이 발전의 끝에 모든 질문을 해결하는 모델이 있으리라는 기대를 하나의 사고실험으로 살펴보고자 한다. 영상 「당신이 수학을 모르는 이유」에서 다룬 자기지시의 발상을 모델 자신의 예측에 적용해 보자.[3]

## 2 마지막 모델에게 던지는 질문

사고실험의 주인공은 모든 질문에 유한한 시간 안에 정답을 내놓는 ‘마지막 모델’  $M$ 이다.  $M$ 은 계산 가능한 소프트웨어이며, 같은 질문에는 같은 답을 낸다고 가정한다. 검색 도구나 다른 모델을 이용한다면 그 전체를  $M$ 으로 보아도 된다.  $M$ 은 “네 답은 옳은가?”라는 질문에도 옳게 답하겠지만, 이 자기 확인만으로는 가정에 문제가 생기지 않는다. 이제 질문을 조금 바꾸어 보자.

모델의 약속을 시험하려면 그 예측과 반대로 행동하는 프로그램 하나면 충분하다.  $M(\langle P, x \rangle)$ 를 “프로그램  $P$ 가 입력  $x$ 에서 종료하는가?”에 대한 답으로 쓰고, 종료하면 1, 종료하지 않으면 0이라고 하자. 프로그램도 소스코드를 입력으로 받을 수 있으므로, 다음과 같이  $P$ 가 자기 소스코드를 입력받는 경우를 묻는 프로그램  $D$ 를 구성할 수 있다.[2]

$$D(P) = \begin{cases} \text{무한히 실행} & M(\langle P, P \rangle) = 1, \\ \text{즉시 종료} & M(\langle P, P \rangle) = 0. \end{cases}$$

$D$ 에 자신의 소스코드를 입력하면 마지막 모델의 두 답변은 모두 빗나간다.  $M$ 이  $D(D)$ 는 종료한다고 답하면  $D(D)$ 는 무한히 실행하고, 종료하지 않는다고 답하면 즉시 종료한다.  $M$ 이 판단을 보류하거나 끝없이 생각한다면 틀린 답은 피할 수 있지만, 모든 질문에 유한한 시간 안에 정답을 준다는 처음의 약속을 지키지 못한다. 계산 가능한  $M$ 을 어떻게 고쳐도 그 모델을 사용하는  $D$ 를 다시 구성할 수 있으므로, 성능 향상만으로 이 약속을 완성할 수는 없다. 결과적으로 전제와 같이 모든 질문에 유한 시간 안에 정답을 내놓는 대규모 언어 모델은 존재할 수 없는 것이다.

### 3 믿지 않기 때문에 믿을 수 있다

OpenAI는 나비에-스토크스 문제의 해결안을 발표하면서 분석적 증명과 Lean 형식화를 공개하고 검증을 수행했다고 밝혔다.[1] 증명 보조기(proof assistant) Lean의 핵심 검사기는 제출된 증명이 정해진 공리와 규칙 아래에서 해당 명제를 뒷받침하는지 확인한다. 이 방식의 장점은 모델이 얼마나 자신 있게 말하는지와 무관하게 증명의 타당성을 다시 검사할 수 있다는 것이다. 모델이 탐색 중 틀렸더라도 최종 증명이 조건을 충족한다면 그 결과를 받아들일 근거가 생긴다.

증명 검사는 모든 질문에 답한다는 약속 없이도 특정 결론을 뒷받침할 수 있다. 공리와 규칙을  $T$ , 명제를  $\varphi$ , 제출된 증명을  $p$ 라 하면 검사의 역할은 다음과 같다.

$$\text{Check}_T(p, \varphi) = \text{통과} \implies T \vdash \varphi.$$

여기서  $T \vdash \varphi$ 는  $T$ 에서  $\varphi$ 가 증명된다는 뜻이다. 특정 증명이 검사에 실패해도 명제가 거짓이라는 뜻은 아니며, 참인 명제 모두에 대해 그 체계 안의 증명을 찾아 준다는 보장도 없다. 따라서 앞의 사고실험과 충돌하지 않는다. 모든 질문에 대한 정답 능력을 보장하려는 목표를, 주어진 형식 체계(formal system)에서 개별 출력의 증거를 확인하는 문제로 바꾼 셈이다.

이렇게 개별 출력의 증거를 확인하는 쪽으로 시선을 옮기면, 그 증거를 마주하고 평가하는 주체로서 인간의 자리가 다시 드러난다. 물론 인공지능의 한계가 곧 인간의 지적 우월성을 뜻하지는 않는다. 인간 역시 모든 질문에 답할 수 없으므로, 모델이 풀지 못한 문제를 언제나 대신 해결할 수는 없다. 우리가 해야 할 일은 기계가 내놓은 출력과 증거를 살피며, 어떤 근거를 신뢰하고 어떤 위험을 감수할지 판단하는 것이다. 체계 안에서 검증된 결과를 현실로 가져와 무엇을 진정한 '올바름'으로 정할지 결정하는 책임은 온전히 인간의 몫으로 남는다.

완벽한 정답을 약속할 수 없는 세계에서, 기계의 답변에 끊임없이 개입하고 문제를 제기하는 이 의심의 여지야말로 앞으로 인간이 맡을 역할이자 살아갈 틈이 될 것이다.

### 참고문헌

- [1] OpenAI. 2026. On the Navier-Stokes Millennium Prize Problem. OpenAI 연구 발표. <https://openai.com/index/navier-stokes-solution/> 분석적 증명과 Lean 형식화 공개.
- [2] Alan M. Turing. 1937. On Computable Numbers, with an Application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society* s2-42, 1 (1937), 230–265. doi:10.1112/plms/s2-42.1.230
- [3] Veritasium 한국어 - 베리타시움. 2021. 당신이 수학을 모르는 이유 (feat. 불완전성의 정리). 유튜브 영상. <https://www.youtube.com/watch?v=oippSXvUlw> 2026년 9월 11일 확인.